# OFFLINE STORAGE SYSTEM

Day 1 Development Notes & Beginner Learning Guide

**What we built:** A local Node.js + Express API that accepts document/audio files, stores the original files on disk, saves their metadata and output in MongoDB, lists records, shows one record, and downloads the original input.

## Development environment

Windows 11 • Node.js v20.9.0 • npm 10.2.3 • MongoDB Server 9.0.4 • mongosh 2.14.1 • VS Code

## Project folder

C:\Users\dhanu\Offline-Storage-System

Prepared from the work completed on 2 November 2026

# 1. What is this project?

The Offline Storage System is a separate application that will receive the original input file and the final output from other processing applications. It stores the original file on the computer and stores searchable record information in MongoDB. Later, a React dashboard will let office users browse history, open a record, and download its input/output.

The two future processing applications are not built yet. We therefore developed and tested this storage service independently using a real PDF and a generated sample WAV file. When the other applications are ready, they can send their input and output to these APIs.

## High-level flow

```
Document or audio application
          |
          |  HTTP multipart upload + output fields
          v
Node.js + Express API (localhost:5000)
          |
          +---- Multer saves original file
          |       backend/uploads/documents/
          |       backend/uploads/audio/
          |
          +---- Mongoose saves record metadata/output
                  MongoDB: offline_storage
```

**Important distinction:** MongoDB stores the record, metadata, and output data. The original PDF/audio bytes are stored as normal files in the uploads folders. This avoids putting large files directly into ordinary MongoDB documents.

## What is complete and what remains?

| Feature | Status at end of Day 1 |
|---|---|
| Node.js / Express backend | Completed and running |
| MongoDB connection | Connected and tested |
| Document upload | Tested with 218.pdf |
| Audio upload | Tested with generated sample.wav |
| List all records | GET API tested; returned 3 records |
| Get one record | GET API tested with PDF record |
| Download original input | PDF download tested successfully |
| Output retrieval API | Next task |
| Search/filter and React dashboard | Not started |
| Integration with the two future processing apps | Later, when those apps are ready |
| Offline office deployment/security hardening | Later |

# 2. Project folders

```
Offline-Storage-System/
■■■ backend/
■   ■■■ server.js
■   ■■■ .env
■   ■■■ package.json
■   ■■■ node_modules/
■   ■■■ logs/
■   ■■■ outputs/
■   ■■■ uploads/
■   ■   ■■■ documents/
■   ■   ■■■ audio/
■   ■■■ src/
■       ■■■ config/
■       ■■■ controllers/
■       ■   ■■■ processingController.js
```

```
■       ■■■ middleware/
■       ■   ■■■ upload.js
■       ■■■ models/
■       ■   ■■■ ProcessingRecord.js
■       ■■■ routes/
■       ■   ■■■ processingRoutes.js
■       ■■■ services/
■■■ frontend/
■■■ deployment/
■■■ documentation/
```

What the main folders mean:

• **backend/** — the server-side application.

• **src/models/** — describes the shape of records stored in MongoDB.

• **src/controllers/** — contains the functions that perform API work.

• **src/routes/** — maps HTTP URLs and methods to controller functions.

• **src/middleware/** — code that runs between the incoming request and controller; Multer is configured here.

• **uploads/** — stores the original input files on disk.

• **frontend/** — reserved for the React dashboard.

• **deployment/** and **documentation/** — reserved for packaging and project documentation.

# 3. Tools and commands used

## Installed backend packages

```
npm init -y
npm install express mongoose cors dotenv helmet multer
npm install --save-dev nodemon
npm list --depth=0
```

| Package | Why we use it |
|---|---|
| express | Creates the HTTP server and API routes. |
| mongoose | Connects Node.js to MongoDB and defines data models. |
| cors | Controls which browser origins can call the API. |
| dotenv | Loads configuration values from the .env file. |
| helmet | Adds common HTTP security headers. |
| multer | Receives multipart file uploads and saves files to disk. |
| nodemon | Development helper that can restart the server when files change; we have not yet configured it as the start command. |

## Useful Windows commands

### Open the backend folder:

```
cd C:\Users\dhanu\Offline-Storage-System\backend
```

### Start the server:

```
node server.js
```

### List middleware files:

```
dir src\middleware
```

### List route files:

```
dir src\routes
```

### Stop a running server:

```
Ctrl + C
```

Keep the backend server running in one Command Prompt window while you test APIs from a second Command Prompt window.

### MongoDB shell

mongosh was extracted as a ZIP and is not currently on the Windows PATH. The working executable path is:

```
"C:\Users\dhanu\Downloads\mongosh-2.14.1-win32-x64\mongosh-2.14.1-win32-x64\bin\mongosh.exe"
```

#### Inside mongosh (not in ordinary CMD):

```
use offline_storage
show collections
db.processingrecords.find().pretty()
```

The commands **use**, **show collections**, and **db...** are MongoDB shell commands. They must be typed after mongosh has opened and shows a prompt such as `offline_storage>`. They are not Windows CMD commands.

# 4. Environment configuration and server

### The .env file

Create the file at **backend/.env**. The filename must be exactly **.env** (not .evn and not .env.txt). We fixed a spelling error during setup.

```
PORT=5000
MONGODB_URI=mongodb://127.0.0.1:27017/offline_storage
```

PORT is the local HTTP port. MONGODB_URI tells Mongoose where the MongoDB server is and which database to use. This is a local connection; the application does not need internet access to connect to MongoDB running on the same computer.

### Current backend/server.js

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
const helmet = require("helmet");
require("dotenv").config();

const processingRoutes = require("./src/routes/processingRoutes");

const app = express();

app.use(helmet());
app.use(cors());
app.use(express.json());

app.use("/api/v1/records", processingRoutes);

app.get("/api/health", (req, res) => {
    res.json({
        status: "success",
        message: "Offline Storage API is running",
        database: mongoose.connection.readyState === 1
            ? "Connected"
            : "Disconnected"
    });
});

const PORT = process.env.PORT || 5000;

async function startServer() {
    try {
        await mongoose.connect(process.env.MONGODB_URI);
        console.log("MongoDB connected successfully");

        app.listen(PORT, () => {
            console.log(`Server running at http://localhost:${PORT}`);
        });
    } catch (error) {
        console.error("MongoDB connection failed:", error.message);
    }
}

startServer();
```

### Line-by-line explanation

• **require(...)** imports a package or another project file.

• **express()** creates the Express application.

• **app.use(...)** installs middleware that runs for incoming requests.

• **express.json()** parses JSON request bodies. File uploads use multipart form data and are handled by Multer.

• **app.use('/api/v1/records', processingRoutes)** mounts the records router under this URL prefix.

• **mongoose.connect(...)** opens the database connection.

• **app.listen(...)** starts accepting HTTP requests on port 5000.

**Health-check URL:**

```
http://localhost:5000/api/health
```

**Verified health-check response:**

```
{
  "status": "success",
  "message": "Offline Storage API is running",
  "database": "Connected"
}
```

# 5. MongoDB record model

File: **backend/src/models/ProcessingRecord.js**. A Mongoose schema defines which fields each processing record contains.

```
const mongoose = require("mongoose");

const processingRecordSchema = new mongoose.Schema(
    {
        processingId: {
            type: String,
            required: true,
            unique: true
        },

        sourceJobId: {
            type: String,
            default: null
        },

        type: {
            type: String,
            enum: ["document", "audio"],
            required: true
        },

        originalFilename: {
            type: String,
            required: true
        },

        inputPath: {
            type: String,
            required: true
        },

        outputFormat: {
            type: String,
            required: true
        },

        outputData: {
            type: mongoose.Schema.Types.Mixed,
            required: true
        },

        status: {
            type: String,
            enum: ["completed", "failed"],
            default: "completed"
        },

        processedAt: {
```

```
            type: Date,
            default: Date.now
        }
    },
    {
        timestamps: true
    }
);

module.exports = mongoose.model(
    "ProcessingRecord",
    processingRecordSchema
);
```

| Field | Meaning |
|---|---|
| processingId | Unique UUID created by our storage service. |
| sourceJobId | Optional job identifier from the original processing application; currently null in our tests. |
| type | Must be either document or audio. |
| originalFilename | The filename supplied by the uploader, e.g. 218.pdf. |
| inputPath | Full local path where Multer saved the original input file. |
| outputFormat | Format label for output, such as json or txt. |
| outputData | The extracted document data or transcript. |
| status | Current record status; our test endpoint currently saves completed records. |
| processedAt | Date/time of processing. |
| timestamps | Mongoose automatically adds createdAt and updatedAt. |

The schema uses **Mixed** for outputData so it can hold different shapes of output. In our current multipart test, JSON was received as a string. We have noted that we should parse JSON output safely in a later improvement so the dashboard can work with structured fields.

# 6. Multer: saving real files

File: **backend/src/middleware/upload.js**. Multer handles incoming multipart/form-data file uploads. We configured separate storage folders for documents and audio.

```
const multer = require("multer");
const path = require("path");
const fs = require("fs");
const crypto = require("crypto");

// Configure storage for uploaded files
function createStorage(folderName) {
    const destination = path.join(
        __dirname,
        "../../uploads",
        folderName
    );

    fs.mkdirSync(destination, { recursive: true });

    return multer.diskStorage({
        destination: (req, file, cb) => {
            cb(null, destination);
        },

        filename: (req, file, cb) => {
            const extension = path.extname(file.originalname);
            const uniqueName = crypto.randomUUID() + extension;

            cb(null, uniqueName);
        }
    });
}

// Document uploads
const documentUpload = multer({
```

```
    storage: createStorage("documents"),
    limits: {
        fileSize: 100 * 1024 * 1024
    }
});

// Audio uploads
const audioUpload = multer({
    storage: createStorage("audio"),
    limits: {
        fileSize: 500 * 1024 * 1024
    }
});

module.exports = {
    documentUpload,
    audioUpload
};
```

### Important concepts

• **path.join()** builds a filesystem path safely from folder segments.

• **__dirname** is the directory containing the current JavaScript file.

• **fs.mkdirSync(..., { recursive: true })** creates the destination directory if it does not already exist.

• **multer.diskStorage()** tells Multer to write the uploaded file to disk.

• **crypto.randomUUID()** generates a unique filename so uploads with identical original filenames do not overwrite each other.

• **fileSize** sets an upload size limit: 100 MiB for documents and 500 MiB for audio.

At this stage, extension/MIME validation has not yet been added. Before real office use, we must validate allowed file types, add authentication/authorization, and consider storage quotas.

## 7. Controller: saving an uploaded file and output

File: **backend/src/controllers/processingController.js**. A controller contains the actual work for an API request. The save function checks required fields, generates an ID, and creates a MongoDB record.

```
const ProcessingRecord = require("../models/ProcessingRecord");
const crypto = require("crypto");

exports.saveProcessingRecord = async (req, res) => {
    try {
        const {
            sourceJobId,
            type,
            outputFormat,
            outputData
        } = req.body;

        if (!req.file) {
            return res.status(400).json({
                status: "error",
                message: "Please upload an input file"
            });
        }

        if (!["document", "audio"].includes(type)) {
            return res.status(400).json({
                status: "error",
                message: "Type must be document or audio"
            });
        }

        if (!outputFormat || outputData === undefined) {
            return res.status(400).json({
                status: "error",
                message: "Output format and output data are required"
            });
        }

        const processingId = crypto.randomUUID();

        const record = await ProcessingRecord.create({
            processingId,
```

```
            sourceJobId,
            type,
            originalFilename: req.file.originalname,
            inputPath: req.file.path,
            outputFormat,
            outputData,
            status: "completed"
        });

        res.status(201).json({
            status: "success",
            message: "File and output saved successfully",
            data: record
        });

    } catch (error) {
        console.error("Storage error:", error);

        if (req.file) {
            const fs = require("fs");

            fs.unlink(req.file.path, (err) => {
                if (err) console.error("File cleanup failed:", err);
            });
        }

        res.status(500).json({
            status: "error",
            message: "Unable to save processing record"
        });
    }
};
```

### How the request moves through the code

**1.** Multer receives the uploaded file before the controller runs.

**2.** Multer writes the file into the documents or audio folder.

**3.** The controller reads text fields from req.body and uploaded file information from req.file.

**4.** A UUID is generated for processingId.

**5.** ProcessingRecord.create() inserts the record into MongoDB.

**6.** The API responds with HTTP 201 (Created) and the saved record.

The cleanup code attempts to delete the uploaded file if MongoDB saving fails, so a failed database insert does not leave an orphaned upload in the normal error path.

# 8. Routes: the API URLs

File: **backend/src/routes/processingRoutes.js**. Routes connect an HTTP method and URL to middleware and a controller.

```
const express = require("express");
const router = express.Router();

const {
    saveProcessingRecord,
    getAllRecords,
    getRecordById,
    downloadInputFile
} = require("../controllers/processingController");

const {
    documentUpload,
    audioUpload
} = require("../middleware/upload");

// Get all processing records
router.get("/", getAllRecords);

// Download original input file
router.get("/:id/input", downloadInputFile);

// Get one record
router.get("/:id", getRecordById);

// Document upload API
```

```
router.post(
    "/documents",
    documentUpload.single("inputFile"),
    (req, res, next) => {
        req.body.type = "document";
        next();
    },
    saveProcessingRecord
);

// Audio upload API
router.post(
    "/audio",
    audioUpload.single("inputFile"),
    (req, res, next) => {
        req.body.type = "audio";
        next();
    },
    saveProcessingRecord
);

module.exports = router;
```

**Route order matters:** put `/:id/input` before `/:id`. Otherwise the general ID route could match the path before the more specific input-download route.

| Method | Endpoint | Purpose |
|---|---|---|
| GET | /api/health | Check backend and database status. |
| POST | /api/v1/records/documents | Upload a document and save its output record. |
| POST | /api/v1/records/audio | Upload an audio file and save its transcript record. |
| GET | /api/v1/records | List all records, newest first. |
| GET | /api/v1/records/:id | Retrieve one record by MongoDB _id. |
| GET | /api/v1/records/:id/input | Download the original uploaded input file. |

# 9. The remaining controller functions

## List all records

```
exports.getAllRecords = async (req, res) => {
    try {
        const records = await ProcessingRecord.find()
            .sort({ createdAt: -1 });

        res.json({
            status: "success",
            totalRecords: records.length,
            data: records
        });

    } catch (error) {
        console.error("Fetch error:", error);

        res.status(500).json({
            status: "error",
            message: "Unable to fetch records"
        });
    }
};
```

**find()** retrieves records. **sort({ createdAt: -1 })** sorts newest first. **res.json()** sends JSON back to the client.

## Get one record by MongoDB ID

```
exports.getRecordById = async (req, res) => {
    try {
        const record = await ProcessingRecord.findById(req.params.id);

        if (!record) {
            return res.status(404).json({
                status: "error",
                message: "Record not found"
```

```
            });
        }

        res.json({
            status: "success",
            data: record
        });

    } catch (error) {
        res.status(400).json({
            status: "error",
            message: "Invalid record ID"
        });
    }
};
```

**req.params.id** is the ID portion of the URL. **findById()** looks up the MongoDB document. A missing record returns 404; a malformed ID returns 400.

### Download original input file

```
const fs = require("fs");
const path = require("path");

// Download original input file
exports.downloadInputFile = async (req, res) => {
    try {
        const record = await ProcessingRecord.findById(req.params.id);

        if (!record) {
            return res.status(404).json({
                status: "error",
                message: "Record not found"
            });
        }

        const filePath = path.resolve(record.inputPath);
        const uploadsDirectory = path.resolve(__dirname, "../../uploads");

        if (
            !filePath.startsWith(uploadsDirectory + path.sep) ||
            !fs.existsSync(filePath)
        ) {
            return res.status(404).json({
                status: "error",
                message: "Original file not found"
            });
        }

        res.download(filePath, record.originalFilename);

    } catch (error) {
        console.error("Download error:", error);

        res.status(500).json({
            status: "error",
            message: "Unable to download file"
        });
    }
};
```

**path.resolve()** converts a path to an absolute path. The check ensures the resolved path is inside the uploads directory. **fs.existsSync()** checks that the file exists. **res.download()** sends it to the browser/client as a download.

# 10. Testing performed today

### A. Start the backend

```
cd C:\Users\dhanu\Offline-Storage-System\backend
node server.js
```

Expected terminal messages:

```
MongoDB connected successfully
Server running at http://localhost:5000
```

### B. Test health endpoint

```
Open in browser:
http://localhost:5000/api/health
```

### C. Upload a PDF

A real PDF named 218.pdf was uploaded from Downloads. Use curl.exe in a separate Windows Command Prompt window:

```
curl.exe -X POST "http://localhost:5000/api/v1/records/documents" ^
  -F "inputFile=@C:\Users\dhanu\Downloads\218.pdf" ^
  -F "outputFormat=json" ^
  -F "outputData={\"name\":\"Test Document\",\"documentType\":\"PDF\"}"
```

The upload succeeded. The record had MongoDB _id `6ae8b7eccd8c2958c91fec30` and processingId `0e5ca43f-2fd8-46fd-84c1-fa7b27b8001a`. The actual PDF was saved in `backend/uploads/documents/` with a UUID filename.

### D. Create a sample WAV file (because no audio file was available)

Windows PowerShell was used to create a short spoken sample WAV using the built-in System.Speech library:

```
Add-Type -AssemblyName System.Speech
$speaker = New-Object System.Speech.Synthesis.SpeechSynthesizer
$speaker.SetOutputToWaveFile("$env:USERPROFILE\Downloads\sample.wav")
$speaker.Speak("This is a sample audio file for testing our offline storage management system.")
$speaker.Dispose()
```

This creates `sample.wav` in the Downloads folder. It is a synthetic test sample, not a real transcription from a production audio engine.

### E. Upload the WAV file

```
curl.exe -X POST "http://localhost:5000/api/v1/records/audio" ^
  -F "inputFile=@C:\Users\dhanu\Downloads\sample.wav" ^
  -F "outputFormat=txt" ^
  -F "outputData=This is a sample audio transcription for testing."
```

The upload succeeded. The audio file was saved under `backend/uploads/audio/`. Its MongoDB _id was `6ae8b8fdcd8c2958c91fec31`.

### F. List all records

```
Open:
http://localhost:5000/api/v1/records
```

The API returned **3 records**: two document records and one audio record. The oldest sample.pdf record came from an earlier API-only test and contains a placeholder inputPath; it is not a real uploaded file.

### G. Retrieve one record

```
http://localhost:5000/api/v1/records/6ae8b7eccd8c2958c91fec30
```

This returned the details for 218.pdf.

### H. Download the original PDF

```
curl.exe -L "http://localhost:5000/api/v1/records/6ae8b7eccd8c2958c91fec30/input" ^
  -o "%USERPROFILE%\Downloads\downloaded-218.pdf"
```

The download completed successfully, and the user confirmed the downloaded PDF opened correctly.

## 11. Important troubleshooting lessons

| Problem | Cause | Fix |
|---|---|---|
| Mongo URI was undefined | The .env file was misspelled as .evn. | Rename it to .env and check its contents. |
| mongosh not recognized | mongosh.exe was extracted but not added to PATH. | Run the executable using its full path. |
| MongoDB commands caused CMD error | Commands such as use are for mongosh, not Windows CMD. | Start mongosh first, then enter Mongo commands. |

| Problem | Cause | Fix |
|---|---|---|
| Cannot find module upload | The middleware file was misspelled uplade.js. | Rename it to upload.js. |
| Cannot GET /api/v1/records | GET route had not been connected/restarted correctly. | Check route import and router.get('/'), then restart server. |
| curl syntax error | curl command was entered inside mongosh. | Run curl.exe in a separate Windows CMD window. |

# 12. Important improvements before real office use

• **Parse JSON output:** multipart fields arrive as strings. When outputFormat is json, safely JSON.parse outputData and return a clear 400 response for invalid JSON. Keep transcript output as plain text.

• **Validate file types:** restrict document uploads to approved PDF/image formats and audio uploads to approved audio formats; do not trust only the file extension.

• **Use sourceJobId/idempotency:** when the other applications integrate, prevent duplicate records if they retry the same job.

• **Improve status handling:** support pending/failed states and ensure a record is marked completed only after both the input and output are safely stored.

• **Authentication and access control:** MongoDB currently reports that access control is disabled. Before office deployment, enable MongoDB authentication and keep the database port private. Add user authentication to the dashboard and APIs.

• **Protect downloads:** the current download handler checks that the resolved file is inside the uploads folder. Before deployment, also add authorization so users can download only records they are allowed to access.

• **Backups and retention:** implement backups, disk-space monitoring, and retention rules.

• **Offline readiness:** package Node dependencies, frontend build, MongoDB installer/service, and any required runtime assets. Test with internet disconnected.

Do not expose MongoDB directly to office users or the LAN. The intended design is for users to call the backend API; the backend alone talks to MongoDB.

# 13. Tomorrow's starting point

**Next task: Step 28 — Output Retrieval API.** Add a GET endpoint that returns the saved output/transcript for a particular record. After that, build search/filter endpoints, then begin the React dashboard.

| Next step | Planned work |
|---|---|
| 28 | GET /api/v1/records/:id/output — retrieve output data. |
| 29 | Improve output handling: parse JSON output into a real object, retain text transcripts. |
| 30 | Search/filter records by type, filename, date and status. |
| 31 | Build React dashboard: history table, record detail view, input download, output view/download. |
| Later | Connect the two future processing applications with small API calls after their processing finishes. |
| Final phase | Secure and package for fully offline office/intranet deployment. |

Resume phrase for the next session: **“Bro, continue Offline Storage System from Step 28 — Output Retrieval API.”**

# 14. Quick self-check questions

**1. What is Express used for?**
It creates the HTTP server and defines API routes.

**2. Why do we use Mongoose?**
It connects Node.js to MongoDB and lets us define schemas and query records.

**3. What does Multer do?**
It receives multipart file uploads and saves the original files to disk.

**4. Why do we store file bytes on disk instead of in the record?**
It keeps large files outside normal MongoDB documents; MongoDB stores the file path and metadata.

**5. What is the difference between req.body and req.file?**
req.body contains submitted text fields; req.file contains metadata about the uploaded file.

**6. What does processingId identify?**
It is a unique identifier generated for a stored processing record.

**7. What is the purpose of GET /api/v1/records?**
It returns the list of stored records.

**8. What does res.download() do?**
It sends a local file as a download response.

**9. Why should the /:id/input route come before /:id?**
The specific path should be matched before the general ID path.

**10. Why is MongoDB authentication needed before office deployment?**
It prevents unrestricted database access and protects stored records.